

# Trabajo Práctico Integrador

ParallelVision  

Pipeline de procesamiento masivo de imágenes con CPU + GPU

**Universidad Nacional de La Matanza (UNLaM)**

**Programación Concurrente**

1.º Cuatrimestre 2026

**Grupo 6**

Fecha de entrega: **08/07/2026**

## Integrantes

| Nombre y Apellido               | DNI        |
|---------------------------------|------------|
| Felice, Tomás Agustín           | 44.789.809 |
| De La Cruz Zamudio, Axel Nahuel | 41.063.583 |
| Graneros, Brian Ariel           | 41.130.084 |

## Repositorio del proyecto

El código fuente completo (backend, frontend, cuaderno Colab, documentación y este informe) se encuentra en el repositorio de GitHub asignado al grupo:

 <https://github.com/UNLAM-PROG-C/2026-PROGC-Q1-M6>

El proyecto de la entrega vive en la carpeta **TP-Integrador/**. El cuaderno de Google Colab (backend CUDA sobre GPU NVIDIA T4) está en **TP-Integrador/colab/ParallelVision\_GPU.ipynb**.

## Índice

### Parte I — Descripción de la solución

1. [Descripción y finalidad](#)
2. [Lenguajes y versiones](#)
3. [Frameworks y librerías principales](#)
4. [Arquitectura](#)
5. [Restricciones de hardware](#)
6. [Sistema operativo y navegador](#)
7. [Puesta en marcha](#)

## Parte II — Manual de usuario

8. [¿Qué es ParallelVision? \(usuario\)](#)
9. [Antes de empezar \(requisitos\)](#)
10. [Instalación](#)
11. [Modo 1 — Dashboard web \(recomendado\)](#)
12. [Modo 2 — Línea de comandos \(CLI\)](#)
13. [Operaciones disponibles](#)
14. [Interpretar los resultados](#)
15. [Preguntas frecuentes y solución de problemas](#)

## Parte III — Conclusiones

16. [Conclusiones](#)

---

# Parte I — Descripción de la solución

---

## 1. Descripción y finalidad

**ParallelVision** es un pipeline de procesamiento masivo de imágenes diseñado para aplicar diversas operaciones y filtros a grandes volúmenes de archivos de forma eficiente.

El problema que resuelve es el **cuello de botella** que se genera al procesar miles de imágenes de manera secuencial. Para solucionarlo, ParallelVision aprovecha al máximo los recursos de hardware disponibles, aplicando técnicas de concurrencia y combinando **procesamiento asíncrono en la CPU** con **aceleración por GPU**.

El usuario apunta el sistema a una carpeta de imágenes, elige una transformación y el sistema la aplica en paralelo sobre todos los archivos. Las operaciones soportadas son:

- **grayscale** — escala de grises.
- **edges** — detección de bordes (Sobel).
- **blur** — desenfoque gaussiano.
- **equalize** — ecualización de histograma.

Dos características centrales lo diferencian:

- **Detección automática de backend en runtime:** al iniciar, la aplicación detecta qué hardware hay disponible y selecciona el mejor motor de procesamiento (CUDA → OpenCL → CPU) sin que el usuario configure nada. Funciona en cualquier máquina, con o sin GPU.
- **Dashboard de speedup en tiempo real:** una interfaz web muestra el progreso imagen por imagen y una comparativa cuantitativa de rendimiento CPU vs GPU, ilustrando de forma tangible el beneficio del paralelismo masivo.

El sistema se puede usar de dos formas: en **modo CLI** (headless, desde la línea de comandos) o a través de un **dashboard web** cliente-servidor.

---

## 2. Lenguajes y versiones

| Componente               | Lenguaje   | Versión                                  |
|--------------------------|------------|------------------------------------------|
| Backend (pipeline + API) | Python     | <b>3.11 o superior</b> (probado en 3.12) |
| Frontend (dashboard)     | TypeScript | <b>~5.7</b> , compilado con Vite         |
| Toolchain frontend       | Node.js    | <b>18 o superior</b> + npm               |

### 3. Frameworks y librerías principales

#### Backend (Python)

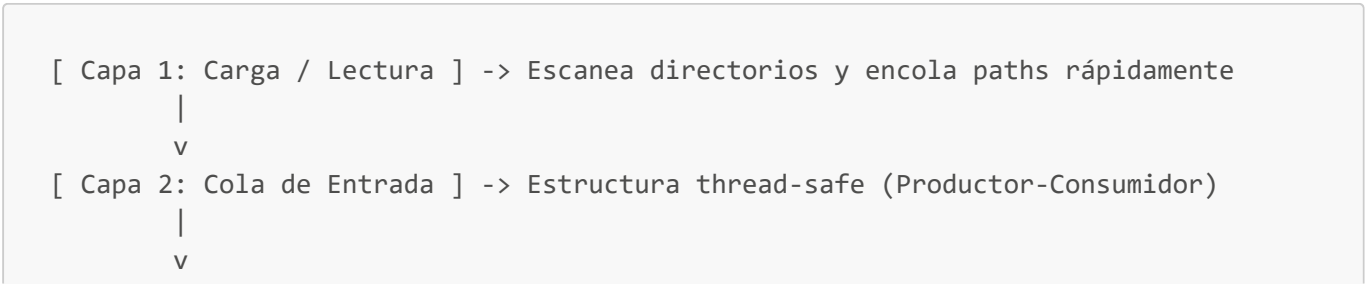
| Área                    | Tecnología                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------|
| API REST + WebSockets   | <b>FastAPI + Uvicorn</b>                                                                  |
| Concurrencia CPU        | <code>concurrent.futures.ThreadPoolExecutor</code>                                        |
| Sincronización          | <code>queue.Queue</code> , <code>threading.Lock</code> , <code>threading.Semaphore</code> |
| Procesamiento de imagen | <b>Pillow, OpenCV, NumPy</b>                                                              |
| Backend GPU NVIDIA      | <b>Numba</b> (kernels CUDA)                                                               |
| Backend GPU AMD / Intel | <b>PyOpenCL</b> (kernels OpenCL)                                                          |
| Calidad de código       | <b>pylint, pytest</b>                                                                     |
| Túnel para Colab        | <b>pyngrok</b>                                                                            |

#### Frontend (dashboard web)

| Área         | Tecnología                                                              |
|--------------|-------------------------------------------------------------------------|
| Framework UI | <b>React 18.3 + Vite 6 + TypeScript</b>                                 |
| Estilos      | <b>TailwindCSS 3.4</b> + componentes estilo <b>shadcn/ui</b> (Radix UI) |
| Gráficos     | <b>Recharts</b> (gráfico de speedup CPU vs GPU)                         |
| Íconos       | lucide-react                                                            |
| Linting      | ESLint 9 + typescript-eslint                                            |

### 4. Arquitectura

El sistema utiliza un diseño estructurado bajo patrones de concurrencia, organizado en 5 capas:



```

[ Capa 3: Workers ] -----> Pool de hilos CPU + Kernels GPU
      |           \
      |           \-----> [ Cola de E/S ] -> [ Hilo de Guardado en Disco ]
      v
[ Capa 4: Agregador ] -----> Recolector de métricas y cálculo de Speedup
      |
      v
[ Capa 5: Dashboard ] -----> API FastAPI + Frontend React (progreso por
WebSocket)

```

## Mapecto de capas a módulos

| Capa             | Responsabilidad                               | Módulos                                                                                                                 |
|------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 1 · Carga        | Escanea la carpeta y encola paths             | <code>pipeline/image_loader.py</code>                                                                                   |
| 2 · Cola         | Buffer thread-safe<br>productor-consumidor    | <code>core/queue_manager.py</code>                                                                                      |
| 3 · Workers      | Pool de hilos CPU + backend<br>GPU + batching | <code>pipeline/worker.py</code> , <code>core/backend/</code> ,<br><code>pipeline/gpu_batcher.py</code>                  |
| 4 ·<br>Agregador | Métricas, speedup y reporte<br>CSV            | <code>pipeline/result_aggregator.py</code> , <code>core/metrics.py</code> ,<br><code>pipeline/report_exporter.py</code> |
| 5 ·<br>Dashboard | API REST/WebSocket + SPA<br>en tiempo real    | <code>api/</code> , <code>frontend/</code>                                                                              |

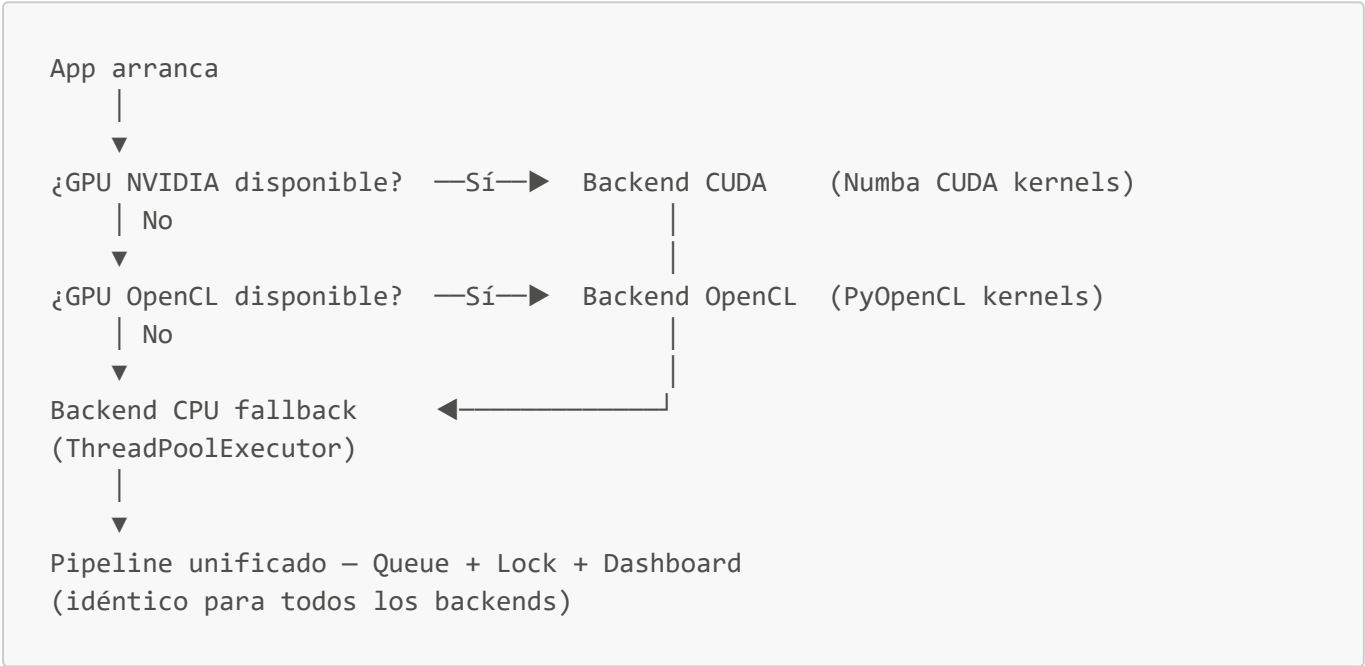
## Conceptos de programación concurrente aplicados

| Concepto                         | Aplicación en ParallelVision                                                                                                                                                                                        |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hilos (Threads)</b>           | Pool de hilos que procesa imágenes en paralelo en la CPU; cada hilo toma un trabajo de la cola y aplica la transformación de forma independiente.                                                                   |
| <b>Productor-Consumidor</b>      | La carga actúa como productor (encola paths); los workers CPU/GPU consumen. A su vez, los workers producen imágenes procesadas hacia una <b>cola de E/S</b> que un hilo de guardado consume para escribir en disco. |
| <b>Mutex / Lock</b>              | El agregador de resultados usa <code>threading.Lock</code> para escritura segura en la estructura compartida de métricas, evitando <i>data races</i> .                                                              |
| <b>Semáforos</b>                 | Un semáforo ( <code>MAX_GPU_CONCURRENT_BATCHES = 2</code> ) limita la cantidad de <b>lotes</b> enviados a la GPU simultáneamente, evitando saturación de memoria de video (OOM).                                    |
| <b>Cola sincronizada (Queue)</b> | Colas <code>queue.Queue</code> thread-safe desacoplan carga, procesamiento y guardado, permitiendo que las etapas corran en paralelo.                                                                               |
| <b>Paralelismo masivo (GPU)</b>  | Kernels CUDA u OpenCL que ejecutan la misma operación sobre miles de píxeles de forma verdaderamente simultánea.                                                                                                    |

| Concepto               | Aplicación en ParallelVision                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------|
| Sincronización CPU-GPU | Manejo explícito de transferencia de datos entre RAM y memoria de video (host-to-device / device-to-host). |

Patrón Strategy para detección de backend

Para lograr la máxima flexibilidad de hardware, el backend implementa el **patrón de diseño Strategy**. Al inicializar el sistema se detecta el hardware disponible y se instancia dinámicamente el backend correspondiente (**CUDABackend**, **OpenCLBackend** o **CPUBackend**). Todos respetan la misma interfaz base **GPUBackend** con el método de entrada **process(image, operation)**. De esta forma, el resto del pipeline (workers y colas) funciona de manera **agnóstica al hardware subyacente**. La detección vive en **core/backend/factory.py** (**get\_backend()**) y las estrategias en **core/backend/cuda.py**, **openc1.py** y **cpu.py**.



5. Restricciones de hardware

El sistema tiene **detección automática de hardware** y utiliza el mejor disponible según la siguiente prioridad:

| Hardware        | Backend        | Librería                  | Prioridad              |
|-----------------|----------------|---------------------------|------------------------|
| NVIDIA GPU      | CUDA           | Numba                     | 1 (más alta)           |
| AMD / Intel GPU | OpenCL         | PyOpenCL                  | 2                      |
| Cualquier CPU   | Multithreading | <b>concurrent.futures</b> | 3 (fallback universal) |

- **No hay requisito duro de GPU:** si no se detecta ninguna, el sistema cae automáticamente al backend **CPU** (ThreadPoolExecutor), que funciona en cualquier máquina.
- **NVIDIA (CUDA):** requiere drivers NVIDIA actualizados y el **CUDA Toolkit** instalado.
- **AMD / Intel (OpenCL):** requiere drivers OpenCL compatibles (incluye GPUs integradas).

- **Forzar CPU (saltar OpenCL):** en GPUs integradas (p. ej. Ryzen) OpenCL puede saturar el procesamiento. Para evitar OpenCL y usar CPU, definir la variable de entorno `PARALLELVISION_DISABLE_OPENCL=1` antes de correr `python main.py` o la API. Solo afecta a OpenCL; la ruta CUDA se mantiene. Para volver al comportamiento normal, no definir la variable.
- **Control de saturación de GPU:** el envío a GPU está limitado por semáforo (`MAX_GPU_CONCURRENT_BATCHES = 2`) y trabaja por lotes (`MAX_GPU_BATCH_SIZE = 32`). Ante un error de memoria de video (OOM) el sistema hace **fallback automático a CPU** por imagen.
- **Entorno GPU alternativo — Google Colab (NVIDIA T4):** se puede correr el backend CUDA en una GPU dedicada de Colab exponiéndolo por túnel ngrok. Ver la guía en `docs/COLAB_GPU.md`.

## 6. Sistema operativo y navegador

### Sistema operativo

Multiplataforma. Probado en:

- **Linux** (distribuciones modernas) y **macOS**.
- **Windows 10 / 11.** El path CUDA en Windows requiere drivers NVIDIA + CUDA Toolkit; el sistema localiza automáticamente las DLLs del Toolkit para Numba (`core/backend/cuda.py`).

El backend CPU y el dashboard funcionan en cualquiera de estos sistemas; la aceleración por GPU depende únicamente de los drivers instalados.

### Navegador

El dashboard es una SPA web que se abre en **cualquier navegador moderno con soporte de WebSocket** — Chrome, Edge o Firefox recientes. No hay una versión mínima fijada. En desarrollo el dashboard se sirve en `http://localhost:5173`.

## 7. Puesta en marcha

### Requisitos previos

- Python **3.11+**
- Node.js **18+** y npm (solo para el dashboard)
- git

### Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/UNLAM-PROG-C/2026-PROGC-Q1-M6.git
cd 2026-PROGC-Q1-M6/TP-Integrador

# 2. Crear y activar el entorno virtual
python -m venv venv
source venv/bin/activate          # Linux / macOS
# .\venv\Scripts\activate         # Windows (PowerShell)
```

```
# 3. Instalar dependencias del backend
pip install -r requirements.txt
```

## Ejecución — Modo CLI (headless)

```
python main.py --input-dir ruta/a/imagenes --operation blur
```

Flags principales: `--operation` (`grayscale` · `edges` · `blur` · `equalize`), `--workers`, `--queue-size`, `--output-dir`, `--report` `archivo.csv`, `--no-save`. Ver `python main.py --help`.

## Ejecución — Modo Dashboard (web)

```
# Terminal 1 — Backend (API + WebSockets) en http://localhost:8000
python -m uvicorn api.main:app --reload

# Terminal 2 — Frontend (dashboard) en http://localhost:5173
cd frontend
npm install
npm run dev
```

En desarrollo, Vite hace de **proxy** de `/api` y `/ws` hacia `http://localhost:8000` (`frontend/vite.config.ts`), por lo que no hace falta configurar CORS.

**Opcional — imágenes de prueba:** `python scripts/descargar_imagenes.py -n 100 -d public/images/in` descarga imágenes aleatorias para probar el sistema.

---

# Parte II — Manual de usuario

---

## 8. ¿Qué es ParallelVision? (usuario)

**ParallelVision** aplica filtros y transformaciones a **grandes volúmenes de imágenes** de forma rápida, aprovechando al máximo el hardware de tu computadora. En lugar de procesar las imágenes una por una, reparte el trabajo en varios hilos de CPU y, si hay una placa de video compatible, la usa para acelerar aún más el procesamiento.

Como usuario solo tenés que hacer tres cosas:

1. Indicar la **carpeta con las imágenes** a procesar.
2. Elegir la **operación** (escala de grises, bordes, desenfoco o ecualización).
3. Presionar **Iniciar** y observar el progreso.

El sistema detecta **solo** el mejor motor de procesamiento disponible (GPU NVIDIA → GPU AMD/Intel → CPU). **No necesitás configurar nada de hardware**: funciona en cualquier máquina, con o sin placa de video.

Hay dos formas de usarlo:

| Modo                 | Para quién                               | Interfaz  |
|----------------------|------------------------------------------|-----------|
| <b>Dashboard web</b> | Uso general, ver progreso y comparativas | Navegador |
| <b>CLI (consola)</b> | Automatización, sin interfaz gráfica     | Terminal  |

## 9. Antes de empezar (requisitos)

Para el uso normal (dashboard o CLI en CPU) solo necesitas:

- **Python 3.11 o superior** (probado en 3.12).
- **Node.js 18 o superior** + npm — *solo* si vas a usar el dashboard web.
- **git** para clonar el repositorio.
- Un **navegador moderno** (Chrome, Edge o Firefox recientes) para el dashboard.

💡 **La GPU es opcional.** Si tu equipo no tiene placa de video compatible, el sistema usa automáticamente la CPU. La aceleración por GPU (NVIDIA/CUDA o AMD-Intel/OpenCL) requiere tener instalados los drivers correspondientes, pero **no es obligatoria**.

## 10. Instalación

Abrí una terminal y ejecutá los siguientes pasos.

```
# 1. Clonar el repositorio
git clone https://github.com/UNLAM-PROG-C/2026-PROGC-Q1-M6.git
cd 2026-PROGC-Q1-M6/TP-Integrador

# 2. Crear y activar el entorno virtual de Python
python -m venv venv
source venv/bin/activate          # Linux / macOS
# .\venv\Scripts\activate         # Windows (PowerShell)

# 3. Instalar las dependencias del backend
pip install -r requirements.txt
```



```
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx2->-r requirements.txt (line 10)) (4.14.0)
Requirement already satisfied: httpcore==2.4.0 in /usr/local/lib/python3.12/dist-packages (from httpx2->-r requirements.txt (line 10)) (2.4.0)
Requirement already satisfied: idna>=3.18 in /usr/local/lib/python3.12/dist-packages (from httpx2->-r requirements.txt (line 10)) (3.18)
Requirement already satisfied: truststore==0.10 in /usr/local/lib/python3.12/dist-packages (from httpx2->-r requirements.txt (line 10)) (0.10.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->-r requirements.txt (line 11)) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->-r requirements.txt (line 11)) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->-r requirements.txt (line 11)) (2026.6.12)
Requirement already satisfied: annotated-types==0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.0->fastapi->-r requirements.txt (line 11)) (0.6.0)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.9.0->fastapi->-r requirements.txt (line 11)) (2.46.4)
Collecting siphash24>=1.6 (from pytools>=2025.1.6->pyopenc1->-r requirements.txt (line 5))
  Downloading siphash24-1.8-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (3.2 kB)
  Downloading pyopenc1-2026.1.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (734 kB)
    734.0/734.0 kB 15.4 MB/s eta 0:00:00
  Downloading pylint-4.0.6-py3-none-any.whl (538 kB)
    538.4/538.4 kB 41.8 MB/s eta 0:00:00
  Downloading pyngrok-8.1.2-py3-none-any.whl (25 kB)
  Downloading astroid-4.0.4-py3-none-any.whl (276 kB)
    276.4/276.4 kB 26.5 MB/s eta 0:00:00
  Downloading isort-8.0.1-py3-none-any.whl (89 kB)
    89.7/89.7 kB 9.3 MB/s eta 0:00:00
  Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
  Downloading pytools-2026.1.1-py3-none-any.whl (99 kB)
    99.2/99.2 kB 10.1 MB/s eta 0:00:00
  Downloading siphash24-1.8-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (103 kB)
    103.2/103.2 kB 11.6 MB/s eta 0:00:00
Installing collected packages: siphash24, pyngrok, mccabe, isort, astroid, pytools, pylint, pyopenc1
Successfully installed astroid-4.0.4 isort-8.0.1 mccabe-0.7.0 pylint-4.0.6 pyngrok-8.1.2 pyopenc1-2026.1.2 pytools-2026.1.1 siphash24-1.8
```

¿No tenés imágenes para probar? El proyecto incluye un script que descarga imágenes de prueba automáticamente:

```
python scripts/descargar_imagenes.py -n 100 -d public/images/in
```

Esto deja 100 imágenes aleatorias en la carpeta `public/images/in`, lista para usar.

## 11. Modo 1 — Dashboard web (recomendado)

El dashboard es una página web que muestra el progreso imagen por imagen y una comparativa de rendimiento **CPU vs GPU** en tiempo real. Funciona con dos procesos: el **backend** (servidor) y el **frontend** (la página).

### 11.1. Iniciar el sistema

Necesitás **dos terminales abiertas** al mismo tiempo, ambas dentro de **TP-Integrador**.

**Terminal 1 — Backend** (servidor de la API, con el entorno virtual activado):

```
python -m uvicorn api.main:app --reload
```

Debe quedar escuchando en `http://localhost:8000`.

**Terminal 2 — Frontend** (la interfaz web):

```
cd frontend
npm install      # solo la primera vez
npm run dev
```

Cuando termine, abrí en el navegador la dirección que indica Vite, normalmente:

```
http://localhost:5173
```

```
(venv) PS C:\Users\tomas\dev\2026-PROGC-Q1-M6\TP-Integrador> python -m uvicorn
n api.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\tomas\\de
v\\2026-PROGC-Q1-M6\\TP-Integrador']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [23724] using WatchFiles
INFO: Started server process [19832]
INFO: Waiting for application startup.
[INFO] Backend seleccionado: OpenCL
[INFO] Backend activo: OpenCL (gfx90c)
INFO: Application startup complete.
```

```
(venv) PS C:\Users\tomas\dev\2026-PROGC-Q1-M6\TP-Integrador\frontend> npm run dev
```

```
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

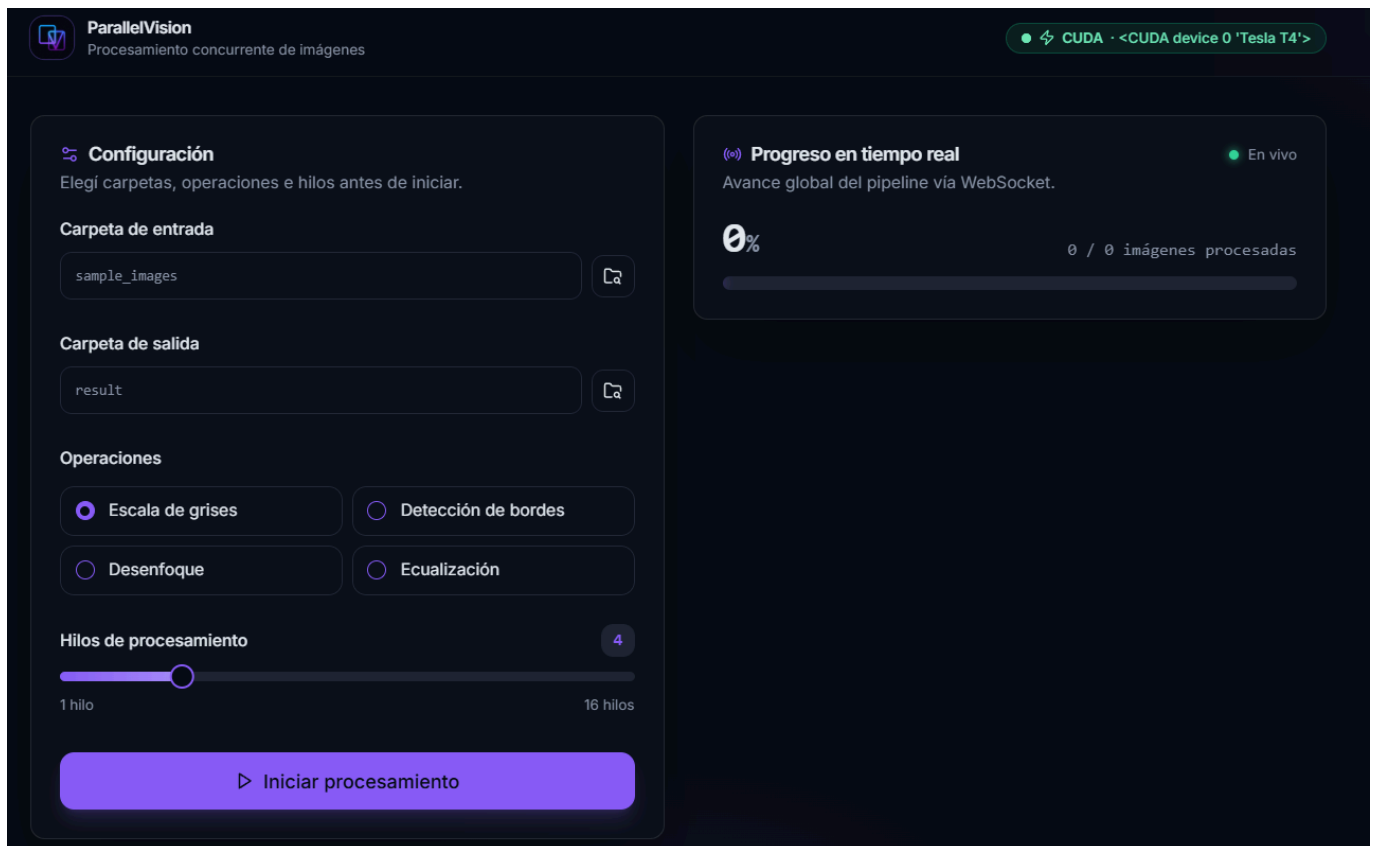
## 11.2. Conocer la pantalla principal

Al abrir el dashboard vas a ver un encabezado con el logo **ParallelVision** y, a la derecha, una **insignia (badge) del backend detectado**:

-  **GPU** (ej. **CUDABackend** • **NVIDIA GeForce...**) si detectó placa de video.
-  **CPU** (ej. **CPUBackend** • **CPU**) si va a usar los hilos del procesador.

Debajo, la pantalla se divide en dos columnas:

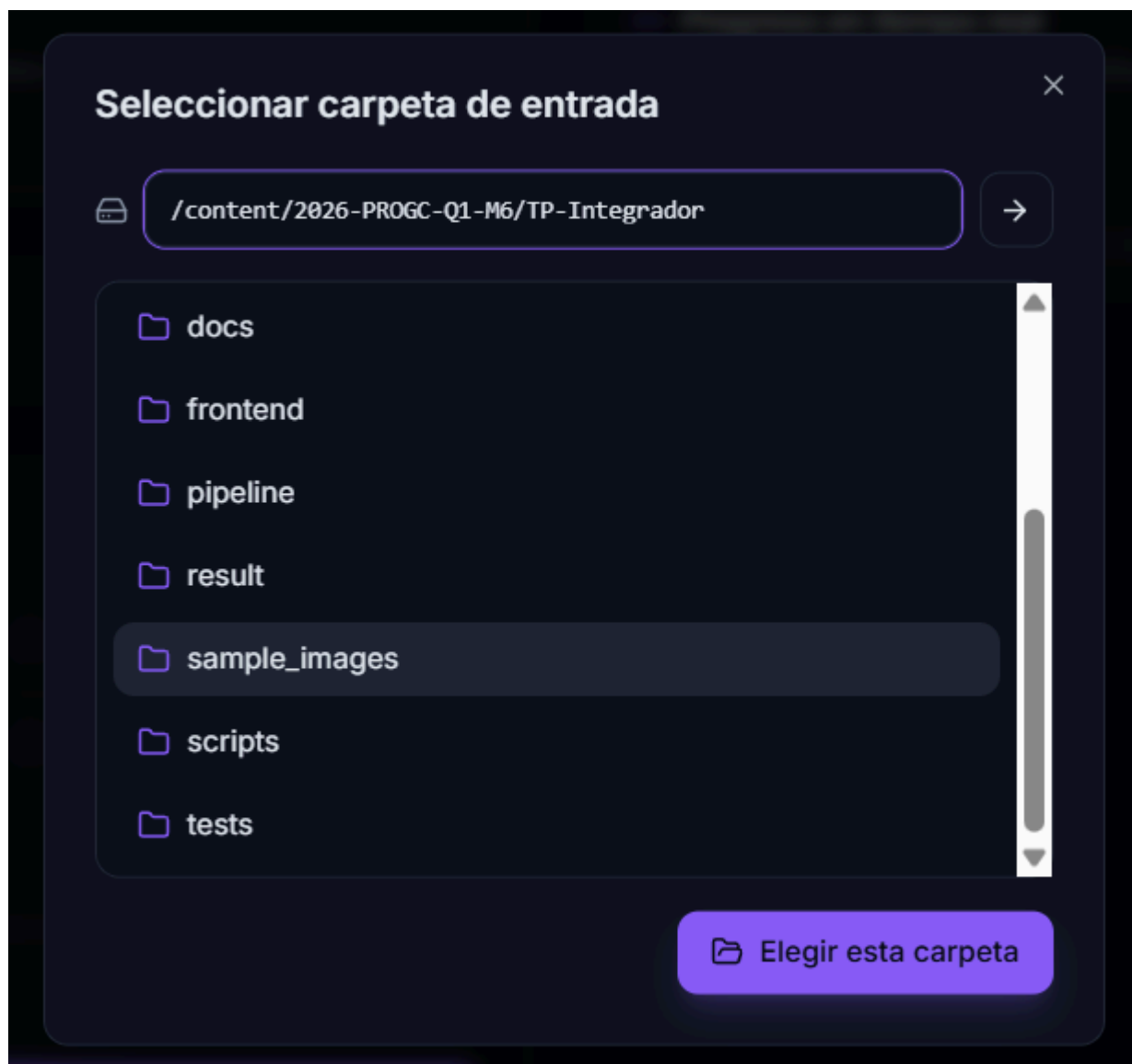
- **Izquierda:** el panel de **Configuración**.
- **Derecha:** el **Progreso en tiempo real**, el **gráfico de speedup** y la **tabla de resultados** (aparecen a medida que corre y termina el procesamiento).



## 11.3. Configurar el procesamiento

En el panel **Configuración** (izquierda), completá los cuatro campos:

1. **Carpeta de entrada** — hacé clic en el campo para abrir el explorador de carpetas. Navegá por el árbol de directorios (o pegá la ruta directamente) y presioná **"Elegir esta carpeta"**. Es la carpeta que contiene las imágenes a procesar.
2. **Carpeta de salida** — de la misma forma, elegí dónde se guardarán las imágenes ya procesadas.
3. **Operaciones** — elegí una de las cuatro transformaciones (ver [sección 13](#)): *Escala de grises*, *Detección de bordes*, *Desenfoque* o *Ecualización*.
4. **Hilos de procesamiento** — deslizá el control para elegir cuántos hilos de CPU usar (de **1 a 16**). Más hilos suele significar más velocidad, hasta el límite de núcleos de tu procesador.



**Configuración**  
Elegí carpetas, operaciones e hilos antes de iniciar.

**Carpeta de entrada**  
/content/2026-PROGC-Q1-M6/TP-Integrador/sample\_images

**Carpeta de salida**  
/content/2026-PROGC-Q1-M6/TP-Integrador/result

**Operaciones**

☒ Escala de grises ☐ Detección de bordes

☐ Desenfoque ☐ Ecualización

**Hilos de procesamiento** 8

1 hilo 16 hilos

▶ Iniciar procesamiento

El botón **"Iniciar procesamiento"** se habilita solo cuando hayas seleccionado **ambas carpetas y una operación**. Si falta algo, aparece el mensaje *"Seleccioná ambas carpetas y al menos una operación."*

#### 11.4. Iniciar y ver el progreso

Presioná ▶ **Iniciar procesamiento**. El panel **Progreso en tiempo real** empieza a actualizarse vía WebSocket:

- Un porcentaje grande (ej. **47 %**) y el contador **current / total imágenes procesadas**.
- Una barra de progreso.
- Un indicador de conexión: ● **"En vivo"** mientras está conectado.

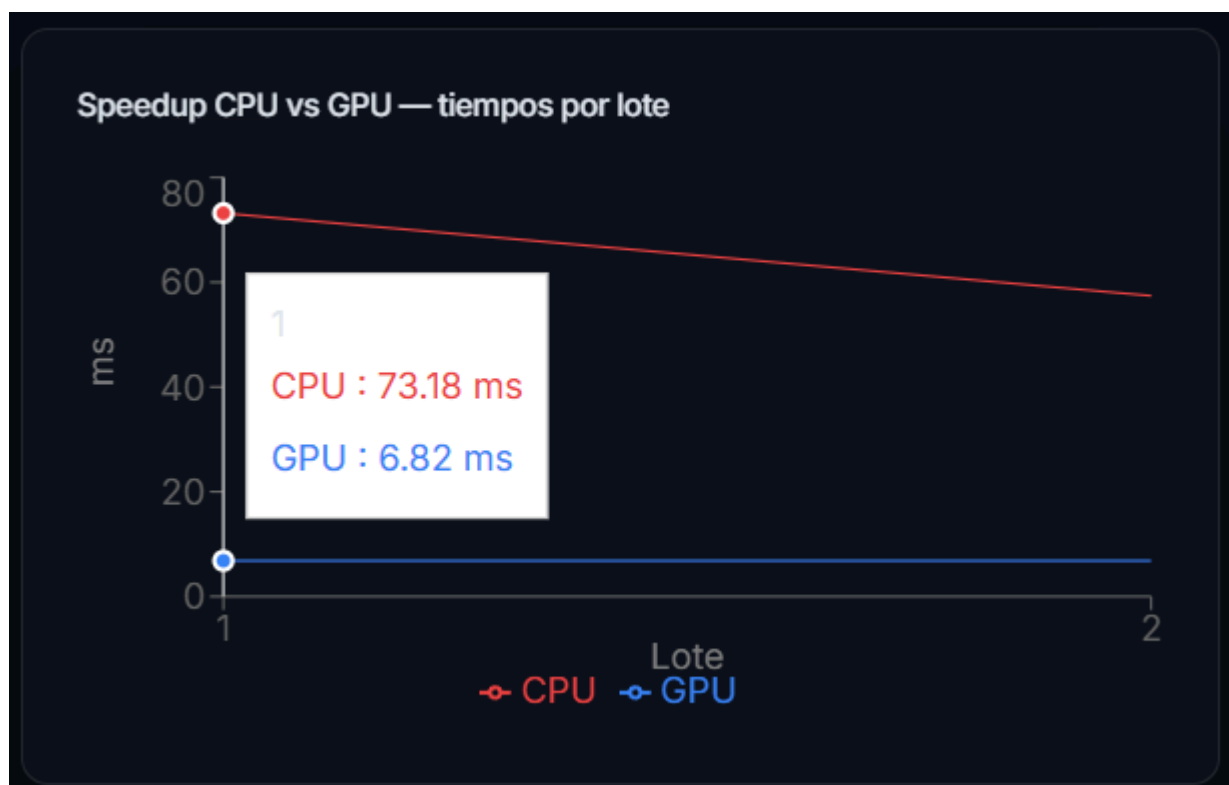


### 11.5. Gráfico de Speedup (CPU vs GPU)

Mientras corre, aparece el gráfico "**Speedup CPU vs GPU — tiempos por lote**". Muestra dos líneas con los milisegundos que tarda cada motor en procesar cada lote de imágenes:

- ● **CPU** (línea roja).
- ● **GPU** (línea azul).

Cuanto **más baja** esté la línea azul respecto de la roja, mayor es la ventaja de la GPU.



### 11.6. Tabla de resultados y exportación a CSV

Al **finalizar** el procesamiento aparece la tabla "**Resultados por operación**" con, para cada operación: **CPU prom (ms)**, **GPU prom (ms)** y **Speedup** (ej. **8.30x**).

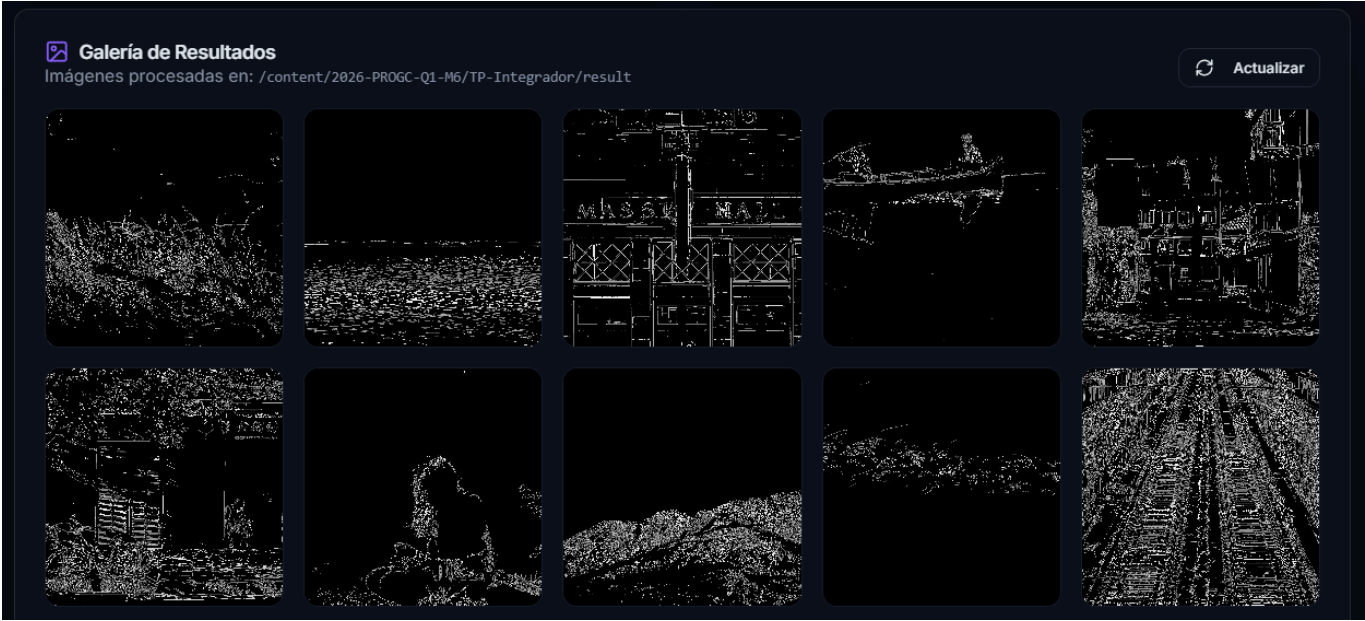
El botón "**Exportar CSV**" descarga estos resultados como archivo **.csv** para conservar el reporte o abrirlo en una planilla.

| Resultados por operación |               |               |         | Exportar CSV |
|--------------------------|---------------|---------------|---------|--------------|
| Operación                | CPU prom (ms) | GPU prom (ms) | Speedup |              |
| edges                    | 65.30         | 6.82          | 9.57x   |              |

11.7. Galería de resultados e inspector

Más abajo, la **Galería de Resultados** muestra las miniaturas de todas las imágenes ya procesadas (se refresca sola al terminar, y también con el botón "**Actualizar**").

Al hacer clic en cualquier miniatura se abre el **Inspector**, con un **comparador "antes / después"**: una barra deslizante para comparar la imagen original con la transformada. Con las flechas ◀▶ navegás entre todas las imágenes.





## 12. Modo 2 — Línea de comandos (CLI)

Si preferís no usar la interfaz web, podés procesar una carpeta directamente desde la terminal (con el entorno virtual activado):

```
python main.py --input-dir ruta/a/imagenes --operation blur
```

### Opciones principales

| Flag                                  | Descripción                                                                                                     | Valor por defecto   |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------|
| <code>--input-dir</code>              | Carpeta con las imágenes a procesar ( <b>obligatorio</b> ).                                                     | —                   |
| <code>--operation</code>              | <code>grayscale</code> · <code>edges</code> · <code>blur</code> · <code>equalize</code> ( <b>obligatorio</b> ). | —                   |
| <code>--workers</code>                | Cantidad de hilos de CPU.                                                                                       | según el sistema    |
| <code>--queue-size</code>             | Tamaño de la cola interna.                                                                                      | valor por defecto   |
| <code>--output-dir</code>             | Carpeta donde guardar los resultados.                                                                           | <code>result</code> |
| <code>--report<br/>archivo.csv</code> | Exporta un reporte de métricas a CSV.                                                                           | —                   |
| <code>--no-save</code>                | Procesa sin guardar las imágenes (solo mide rendimiento).                                                       | desactivado         |

Para ver la ayuda completa:

```
python main.py --help
```

### Ejemplo de salida

Al terminar, la CLI imprime el backend usado, la cantidad de imágenes, el tiempo total, el *throughput* (imágenes por segundo) y un resumen por operación:

```
[INFO] Backend activo: CUDABackend
[INFO] Procesadas 100 imágenes en 3.42 s (29.24 img/s)

Resumen por operación:
  blur → count=100  avg=12.3 ms  min=9.8 ms  max=21.1 ms
```

```
!python main.py --input-dir sample_images/ --operation blur --output-dir result/

[INFO] init
[INFO] Backend seleccionado: CUDA
[INFO] Hilo de I/O iniciado (Guardado de imágenes)
[INFO] Imágenes encontradas: 100
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 4 will likely result in GPU under-util
  warn(errors.NumbaPerformanceWarning(msg))
[INFO] add pending dealloc: cuMemFree 3072 bytes
[INFO] add pending dealloc: cuMemFree 3072 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatcher.py:696: NumbaPerformanceWarning: Grid size 8 will likely result in GPU under-util
  warn(errors.NumbaPerformanceWarning(msg))
[INFO] add pending dealloc: cuMemFree 6144 bytes
[INFO] add pending dealloc: cuMemFree 6144 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
[INFO] Rutas encoladas; sentinelas: 4
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] dealloc: cuMemFree 3072 bytes
[INFO] dealloc: cuMemFree 3072 bytes
[INFO] dealloc: cuMemFree 100 bytes
[INFO] dealloc: cuMemFree 6144 bytes

[INFO] dealloc: cuMemFree 6144 bytes
[INFO] dealloc: cuMemFree 6144 bytes
[INFO] dealloc: cuMemFree 100 bytes
[INFO] dealloc: cuMemFree 199065600 bytes
[INFO] dealloc: cuMemFree 199065600 bytes
[INFO] dealloc: cuMemFree 100 bytes
[INFO] dealloc: cuMemFree 199065600 bytes
[INFO] dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 199065600 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
[INFO] add pending dealloc: cuMemFree 24883200 bytes
[INFO] add pending dealloc: cuMemFree 24883200 bytes
[INFO] add pending dealloc: cuMemFree 100 bytes
[INFO] Agregador de resultados finalizado
[INFO] Manifest JSON guardado en result/manifest.json
[INFO] Hilo de I/O finalizado
[INFO] Backend activo: CUDABackend
[INFO] Procesadas 100 imágenes en 5.91 s (16.93 img/s)

Resumen por operación:
  blur → count=100  avg=21.0 ms  min=10.0 ms  max=32.8 ms
```

## 13. Operaciones disponibles

| Operación | Nombre en el dashboard | Qué hace                                            |
|-----------|------------------------|-----------------------------------------------------|
| grayscale | Escala de grises       | Convierte la imagen a blanco y negro.               |
| edges     | Detección de bordes    | Resalta los contornos (filtro Sobel).               |
| blur      | Desenfoque             | Aplica un desenfoque gaussiano (suaviza la imagen). |



| Operación             | Nombre en el dashboard | Qué hace                                       |
|-----------------------|------------------------|------------------------------------------------|
| <code>equalize</code> | <b>Ecualización</b>    | Mejora el contraste ecualizando el histograma. |

## 14. Interpretar los resultados

- **Speedup (ej. 8.30x):** cuántas veces más rápido fue el motor GPU frente a la CPU para esa operación. Un 4.5x significa "4,5 veces más rápido".
- **CPU prom / GPU prom (ms):** tiempo promedio, en milisegundos, que tardó cada motor en procesar una imagen (o lote).
- **Throughput (img/s):** cantidad de imágenes procesadas por segundo (en la CLI).
- **Sin GPU:** si el sistema corre en CPU, la columna GPU y el speedup muestran `—`, lo cual es esperable y correcto.

Las imágenes procesadas quedan en la **carpeta de salida** que elegiste, junto con un archivo `manifest.json` que el dashboard usa para armar la galería.

## 15. Preguntas frecuentes y solución de problemas

**El badge dice "CPU" y esperaba GPU.** El sistema no detectó una placa compatible o faltan drivers. Con NVIDIA se necesitan los drivers + CUDA Toolkit; con AMD/Intel, drivers OpenCL. Igual funciona todo en CPU.

**El procesamiento con GPU integrada (ej. Ryzen) se satura o va lento.** Las GPU integradas exponen OpenCL pero no rinden como una placa dedicada y pueden saturar el pipeline. Para forzar el uso de CPU y saltar OpenCL, definí la variable de entorno `PARALLELVISION_DISABLE_OPENCL=1` antes de iniciar el backend o la CLI:

```
# Windows (PowerShell)
$env:PARALLELVISION_DISABLE_OPENCL = "1"; python main.py --input-dir
ruta/a/imagenes
```

```
# Linux / macOS
PARALLELVISION_DISABLE_OPENCL=1 python main.py --input-dir ruta/a/imagenes
```

Solo afecta a OpenCL (la ruta CUDA de NVIDIA se mantiene). Para volver al comportamiento normal, no defines la variable (o poné `=0`).

**El botón "Iniciar procesamiento" está deshabilitado.** Faltó seleccionar la carpeta de entrada, la de salida o la operación. Completá los tres.

**Aparece "Sin imágenes en el directorio" o "Directorio no encontrado".** La carpeta de entrada no existe o no contiene imágenes válidas. Verificá la ruta, o usá el script de descarga de imágenes de prueba (ver [sección 10](#)).

**"Pipeline ya en ejecución".** Ya hay un procesamiento en curso. Esperá a que termine antes de iniciar otro.

La galería dice que no encuentra `manifest.json`. El procesamiento todavía no terminó o la carpeta de salida es incorrecta. Esperá a que finalice y presioná "Actualizar".

El dashboard no carga / no conecta. Verificá que ambas terminales estén corriendo (uvicorn en :8000 y Vite en :5173) y que abriste la dirección que indicó Vite.

---

## Parte III — Conclusiones

---

### 16. Conclusiones

En esta sección describimos los aprendizajes y, sobre todo, las **dificultades** que encontramos al realizar el trabajo práctico, fundamentadas en el código del repositorio.

#### 16.1. Detección automática de hardware

La mayor dificultad conceptual fue lograr que **el mismo código corriera en cualquier máquina**. Desde el propio código se puede interrogar a la máquina para saber qué GPU tiene —o si no tiene ninguna— y ofrecer aceleración de forma transparente. La detección (`core/backend/factory.py`) sigue el orden **CUDA** → **OpenCL** → **CPU** en capas cada vez más específicas:

1. **Import guard**: si la librería (`numba`, `pyopencl`) no está, un `except ImportError` descarta el backend sin intentar usarlo.
2. **Consulta al driver**: se verifica que exista hardware real (`cuda.is_available()`, `cl.get_platforms()`).
3. **Excepciones específicas**: ante un error conocido del driver se registra y se **degrada al siguiente backend** en lugar de abortar.

Los casos de borde reales fueron los que más nos costaron: un parche de rutas del CUDA Toolkit en Windows y un *fallback* al "CUDA Simulator". Aprendimos que detectar hardware es tanto consultar APIs como manejar los entornos donde esas APIs fallan.

#### 16.2. Impacto del diseño GPU en el rendimiento

Descubrimos que **tener GPU no garantiza velocidad: hay que saber aprovecharla**. Una implementación ingenua puede ser más lenta que la CPU. Dos decisiones fueron clave:

- **Grid 3D para batches**: los kernels batcheados usan un grid (`H, W, N`) (`_get_batch_grid_dims` en `core/backend/cuda.py`), de modo que **un solo lanzamiento procesa N imágenes** y llena de trabajo los miles de hilos del dispositivo.
- **Amortizar la transferencia**: el costo dominante no es calcular, sino **mover datos por el bus PCIe**. Por eso `process_batch()` apila las N imágenes y hace **una sola transferencia por lote**. El `GpuBatcher` (`pipeline/gpu_batcher.py`) acumula imágenes del mismo shape y dispara el lote al llenarse.

La lección central: si por cada imagen hacés un lanzamiento de kernel y un par de transferencias, el overhead se come la ventaja de la GPU.

#### 16.3. Concurrencia — elegir la primitiva correcta

El pipeline nos obligó a usar varios mecanismos distintos, cada uno para un problema puntual:

- **Cola acotada** (`ImageQueue`): el `maxsize` da **contrapresión** natural —si los consumidores no dan abasto, el productor se bloquea en vez de agotar la memoria.
- **Semáforo** (`_gpu_semaphore`): limita a 2 los lotes GPU simultáneos para **no saturar la VRAM**.
- **Lock** (`MetricsCollector`): protege el estado escrito por muchos hilos; las lecturas **copian dentro del lock y calculan afuera** para retenerlo lo mínimo.
- **Thread-Local Storage** (`_COMPUTE_TLS`, `_FALLBACK_TLS`): estado por hilo sin sincronizar. En vez de proteger lo compartido, **eliminamos el hecho de compartirlo**.
- **Sentinela / poison pill**: el `ResultAggregator` termina al recibir `None`; con `task_done()/join()` logra un **apagado ordenado**.
- **Hilo de E/S dedicado** (`ImageSaver`): desacopla el guardado a disco para que los workers de cómputo no se frenen esperando al disco.

La conclusión: **no existe "la" primitiva de concurrencia**; la habilidad está en elegir la adecuada (cola, semáforo, lock, TLS, sentinela) para cada problema. Depurar condiciones de carrera fue una de las tareas más laboriosas del trabajo.

## 16.4. Trabajo coordinado en equipo

Buena parte de la dificultad fue de **proceso**:

- **Estándares escritos**: una guía de estilo común con reglas duras (máximo 15 líneas por función, sin números mágicos, docstrings) volvió objetivas las revisiones.
- **Git disciplinado**: GitHub Flow con `develop` como rama de integración, **PRs con revisión aprobada** y Conventional Commits, lo que nos permitió **verificarnos mutuamente** el código.
- **Arquitectura temprana**: definir las **5 capas** y el contrato `GPUBackend` dio interfaces estables para avanzar en paralelo sin bloquearnos.

La coordinación no se improvisa: los estándares, los PRs y la arquitectura acordada al inicio fueron la infraestructura del trabajo en paralelo.

## 16.5. Patrones de diseño

Aplicamos patrones resolviendo problemas reales: **Strategy** (`GPUBackend` con CPU/CUDA/OpenCL intercambiables), **Factory** (`get_backend()` encapsula la detección) y **Producer-Consumer** (colas thread-safe entre capas). Dos consecuencias valiosas:

1. **Fallback resiliente en caliente**: si un lote agota la VRAM, `process()` no falla —captura la excepción, degrada esa imagen a CPU (`_handle_oom`) y la etiqueta como `cpu-fallback`. La abstracción de backend lo hizo trivial.
2. **Extensibilidad demostrada**: agregar OpenCL **no requirió tocar el pipeline**, solo implementar la interfaz. Es la prueba concreta del valor de Strategy.

## 16.6. Conclusión general

Tres pilares resumen el trabajo:

1. **Concurrencia bien modelada** — la primitiva correcta para cada problema.

2. **GPU bien aprovechada** — el hardware paralelo solo rinde si se lo alimenta bien (batching, grid 3D, transferencias amortizadas).
3. **Proceso de equipo disciplinado** — estándares, revisión por PRs y arquitectura acordada de antemano.

El aprendizaje central: **la performance no es un accidente del hardware, sino una consecuencia del diseño**. La misma GPU puede acelerar diez veces o frenar el sistema según cómo se la use.